Chatbot IA Móvil

Documento de arquitectura y plan de desarrollo

Estudiante: _____

Materia: _____

Docente: _____

Proyecto: Aplicación móvil individual

Fecha de entrega: 29 de septiembre de 2026

Contenido

1. Caso de estudio 1
1.1 Actores 1
1.2 Requerimientos funcionales 1
1.3 Requerimientos no funcionales 1
2. Plan del proceso de desarrollo (Scrum) 2
2.1 Roles 2
2.2 Artefactos y eventos 2
2.3 Product Backlog 2
2.4 Cronograma de sprints 3
3. Justificación de la arquitectura 3
3.1 Por qué un backend propio 3
3.2 Capas del sistema 4
3.3 Alternativas descartadas 4
4. Diagrama de la arquitectura 5
4.1 Diagrama de componentes 5
4.2 Secuencia: enviar un mensaje al chatbot 6
4.3 Modelo de datos 6
5. Propuesta de patrones de diseño 7
6. Justificación de los frameworks 7
6.1 Framework móvil: comparación 7
6.2 Frameworks y librerías del proyecto 8
7. Esquema de pruebas 8
7.1 Niveles de prueba 9
7.2 Casos de prueba 9
8. Estrategia de versionamiento 10
8.1 Ramas 10
8.2 Versiones semánticas: MAYOR.MENOR.PARCHE+BUILD 10
8.3 Commits convencionales 11
8.4 Qué no se versiona 11
9. Configuración del entorno 11
9.1 Herramientas utilizadas 11
9.2 Parámetros del servidor (server/.env) 12
9.3 Parámetros de la app 12
9.4 Pasos para levantar el entorno 12

1. Caso de estudio

Chatbot IA es una aplicación móvil Android que permite a usuarios registrados conversar con un asistente de inteligencia artificial, guardar su historial y, en el caso del administrador, gestionar las cuentas de usuario.

Problema. Los usuarios necesitan un asistente conversacional accesible desde el celular, con historial propio y privado. La organización necesita controlar quién accede y no exponer la clave del proveedor de IA dentro de la app.

Objetivo general. Desarrollar una app móvil con Flutter conectada a un backend propio que autentique usuarios, almacene conversaciones en una base de datos y obtenga respuestas de un modelo de lenguaje (ChatGPT u otro proveedor configurable).

1.1 Actores

Actor	Descripción	Permisos
Usuario	Persona registrada que usa el chat	Registrarse, iniciar sesión, chatear, ver y borrar sus conversaciones
Administrador	Usuario con rol `admin`, creado al iniciar el servidor	Todo lo del usuario + CRUD de usuarios y asignación de roles
Proveedor de IA	Servicio externo (OpenAI / Anthropic)	Genera las respuestas del asistente

1.2 Requerimientos funcionales

ID	Requerimiento
RF-01	Registro de usuario con nombre, usuario y contraseña (mínimo 6 caracteres)
RF-02	Inicio y cierre de sesión con token JWT persistente en el dispositivo
RF-03	Enviar mensajes y recibir respuestas de la IA con el contexto de la conversación
RF-04	Listar, abrir y crear conversaciones; título automático con el primer mensaje
RF-05	Borrar una conversación o todas las del usuario
RF-06	CRUD de usuarios (crear, listar, editar, eliminar) solo para administradores
RF-07	Crear automáticamente un usuario administrador con contraseña al iniciar el servidor
RF-08	Persistir usuarios, conversaciones y mensajes en una base de datos

1.3 Requerimientos no funcionales

ID	Requerimiento
RNF-01	Seguridad: contraseñas con hash bcrypt; clave de IA solo en el servidor
RNF-02	Rendimiento: inicio de sesión menor a 0.5 s en red local (medido: 0.1 s)
RNF-03	Integridad: siempre debe existir al menos un administrador

ID	Requerimiento
RNF-04	Usabilidad: Material Design 3, modo claro y oscuro, mensajes de error en español
RNF-05	Mantenibilidad: proveedor de IA intercambiable por configuración

2. Plan del proceso de desarrollo (Scrum)

Se usa Scrum adaptado a un proyecto individual: cinco sprints cortos de 1 a 2 días que terminan con la entrega el 29 de septiembre de 2026. Scrum se eligió porque entrega un incremento funcional en cada sprint y permite ajustar el alcance cuando aparecen problemas, como ocurrió con la librería de base de datos y el proveedor de IA.

2.1 Roles

Al ser un trabajo individual, el estudiante asume los tres roles; el docente actúa como cliente que valida el incremento.

Rol	Responsable	Responsabilidad
Product Owner	Estudiante	Prioriza el backlog según los requerimientos del caso
Scrum Master	Estudiante	Cuida los tiempos de cada sprint y elimina bloqueos
Equipo de desarrollo	Estudiante	Diseña, programa y prueba cada incremento
Cliente / stakeholder	Docente	Revisa el incremento y aprueba la entrega

2.2 Artefactos y eventos

- **Product Backlog:** historias de usuario priorizadas (tabla 2.3).
- **Sprint Backlog:** historias elegidas para cada sprint, divididas en tareas.
- **Incremento:** versión funcional etiquetada en Git al cierre de cada sprint (ver sección 8).
- **Definición de terminado:** código en `develop`, `flutter analyze` sin errores, pruebas pasando y funcionalidad probada en el dispositivo.
- **Eventos:** planificación al inicio de cada sprint (30 min), revisión diaria de avance (10 min), revisión del incremento y retrospectiva al cierre (30 min).

2.3 Product Backlog

ID	Historia de usuario	Prioridad	Puntos	Sprint
HU-01	Como usuario quiero registrarme para tener una cuenta propia	Alta	3	1
HU-02	Como usuario quiero iniciar sesión y que la app recuerde mi sesión	Alta	3	1
HU-03	Como administrador quiero que exista una cuenta admin desde el inicio	Alta	2	1
HU-04	Como usuario quiero enviar mensajes y recibir respuestas de la IA	Alta	5	2
HU-05	Como usuario quiero ver mi historial de conversaciones	Media	3	2

ID	Historia de usuario	Prioridad	Puntos	Sprint
HU-06	Como usuario quiero borrar una o todas mis conversaciones	Media	2	2
HU-07	Como administrador quiero crear, editar y eliminar usuarios	Alta	5	3
HU-08	Como administrador quiero asignar el rol de admin a otros usuarios	Media	2	3
HU-09	Como usuario quiero ver errores claros cuando la IA no responde	Baja	2	3
HU-10	Como docente quiero la documentación de arquitectura y pruebas	Alta	3	4

2.4 Cronograma de sprints

Sprint	Fechas	Objetivo	Historias	Entregable
0	21–22 sep	Configurar entorno, repositorio y diseño de arquitectura	—	Proyecto Flutter y servidor base (v0.1.0)
1	23–24 sep	Autenticación y base de datos	HU-01, HU-02, HU-03	Login y registro funcionando (v0.2.0)
2	25–26 sep	Chat con IA e historial	HU-04, HU-05, HU-06	Chat conectado a ChatGPT (v0.3.0)
3	27 sep	Administración de usuarios y manejo de errores	HU-07, HU-08, HU-09	Panel de administrador (v0.4.0)
4	28 sep	Pruebas, correcciones y documentación	HU-10	Versión candidata (v1.0.0)
Entrega	29 sep	Presentación al docente	—	APK + código + este documento

3. Justificación de la arquitectura

Se eligió una arquitectura cliente-servidor de tres capas: app Flutter (presentación), API REST en Node.js (lógica de negocio) y SQLite (datos), con el proveedor de IA como servicio externo consumido solo por el servidor.

3.1 Por qué un backend propio

- **Seguridad de la clave:** una clave de API dentro del APK puede extraerse descompilando la app. En el servidor queda en `server/.env`, fuera del repositorio.
- **Control de acceso:** el servidor valida el token JWT y el rol en cada petición; un usuario no puede leer conversaciones ajenas ni usar el CRUD de usuarios.
- **Datos centralizados:** el historial vive en la base de datos, no en el teléfono, y el administrador gestiona cuentas desde un solo lugar.
- **Proveedor intercambiable:** cambiar de ChatGPT a Claude es una variable de entorno (`AI_PROVIDER`); la app no se recompila.

3.2 Capas del sistema

Capa	Tecnología	Responsabilidad	Archivos principales
Presentación	Flutter (Dart)	Pantallas, navegación, estado de sesión	`lib/screens/*.dart`, `lib/main.dart`
Acceso a servicios (cliente)	`http` + `shared_preferences`	Llamadas REST, token persistente, errores	`lib/api_service.dart`
API / controladores	Express 5	Rutas REST, validación de entrada	`server/src/routes/*.js`
Seguridad	JWT + bcryptjs	Autenticación y autorización por rol	`server/src/auth.js`
Servicio de IA	SDK `openai` / `@anthropic-ai/sdk`	Generar respuestas con el historial	`server/src/ai.js`
Datos	SQLite (`node:sqlite`)	Tablas, integridad referencial, admin inicial	`server/src/db.js`

3.3 Alternativas descartadas

Alternativa	Motivo del descarte
App sin backend (IA llamada directo desde Flutter)	Expone la clave de API y no permite CRUD de usuarios centralizado
Firebase (Auth + Firestore)	Dependencia de un servicio externo y menor control del modelo de datos relacional pedido
Microservicios	Sobredimensionado para un proyecto individual de una semana

4. Diagrama de la arquitectura

4.1 Diagrama de componentes

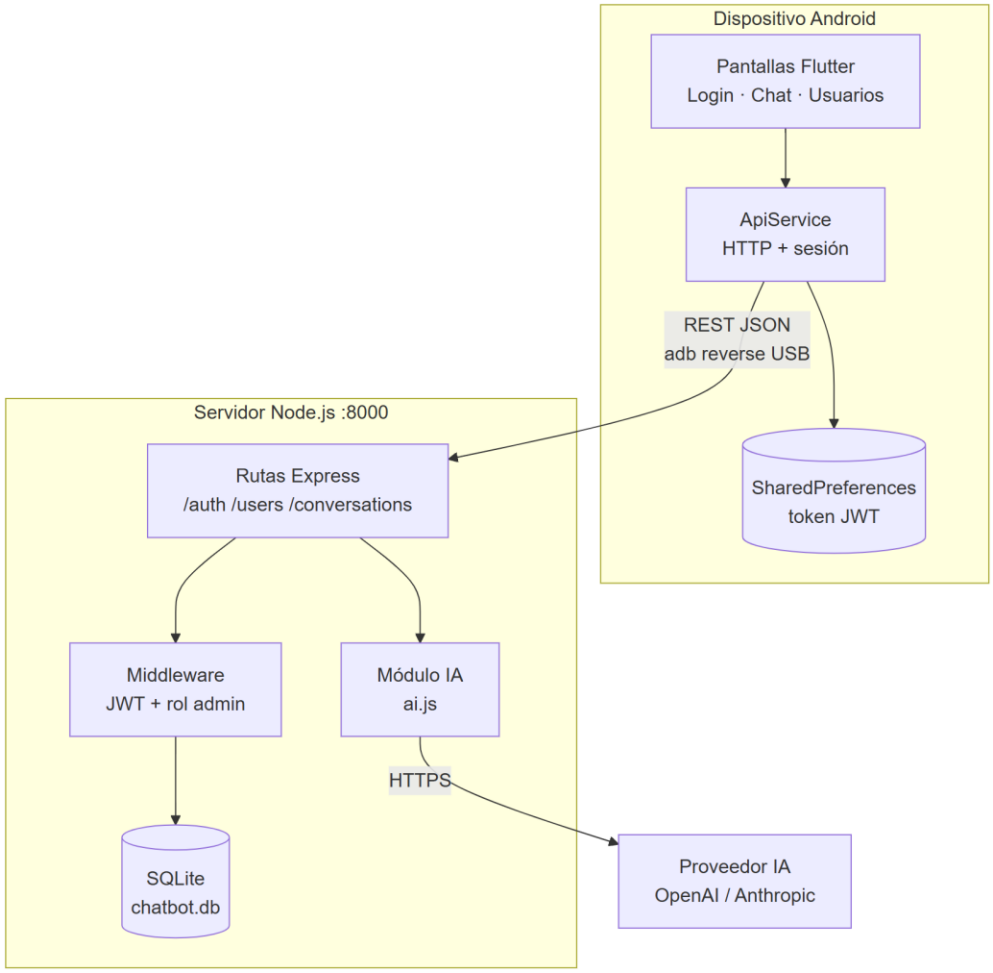


Figura 1. Componentes del sistema y comunicación entre capas

La app solo conoce la URL del servidor; la base de datos y la clave del proveedor de IA nunca salen del servidor.

4.2 Secuencia: enviar un mensaje al chatbot

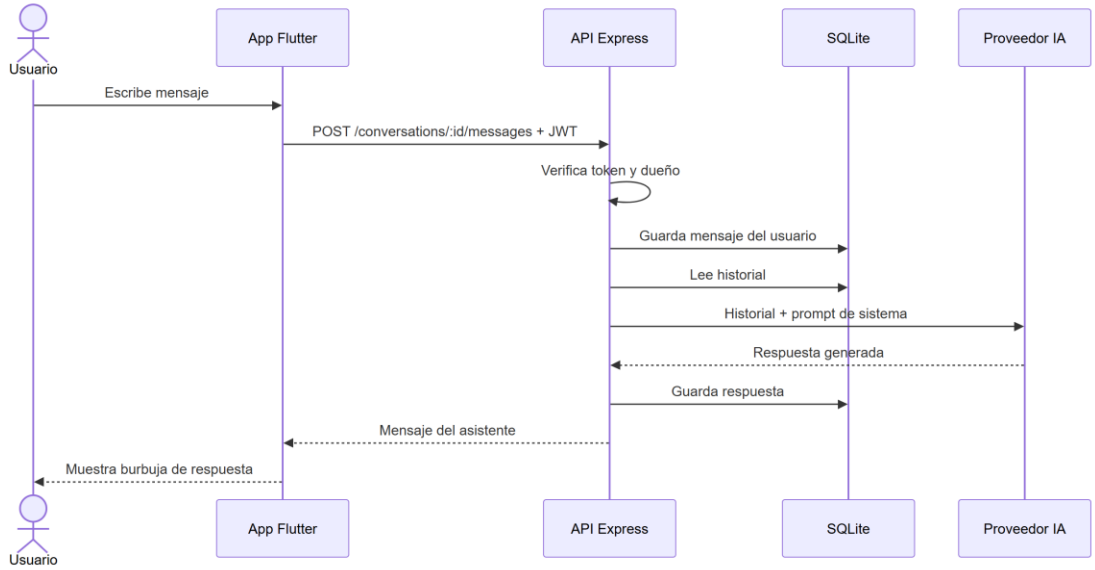


Figura 2. Flujo de un mensaje desde el usuario hasta el proveedor de IA

Si el proveedor falla, el servidor borra el mensaje del usuario y devuelve un error claro (sin créditos, clave inválida o límite de solicitudes).

4.3 Modelo de datos

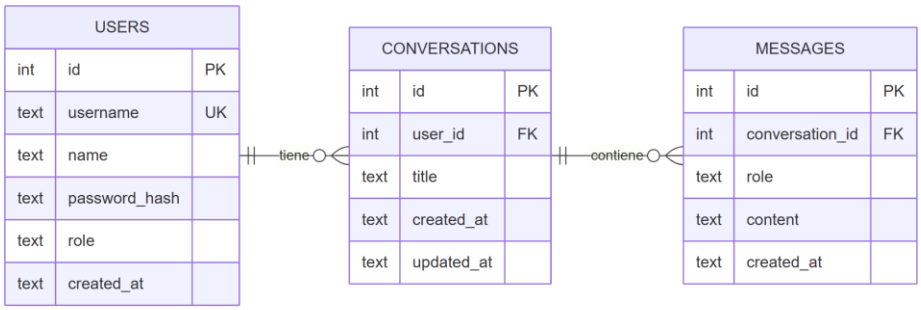


Figura 3. Diagrama entidad-relación de la base de datos SQLite

Tabla	Clave foránea	Regla al borrar	Restricciones
`users`	—	Borra sus conversaciones	`username` único sin distinguir mayúsculas; `role` en (`user`, `admin`)
`conversations`	`user_id → users.id`	Borra sus mensajes	Índice por `user_id`
`messages`	`conversation_id → conversations.id`	—	`role` en (`user`, `assistant`); índice por conversación

5. Propuesta de patrones de diseño

Se aplican seis patrones, elegidos para mantener la app simple sin librerías extra de estado y para aislar el proveedor de IA del resto del sistema.

Patrón	Tipo	Dónde se aplica	Beneficio
Singleton	Creacional	`ApiService.instance` en Flutter; conexión db y clientes de IA en el servidor	Una sola sesión y una sola conexión compartidas por toda la app
Observer	Comportamiento	`ApiService` extiende `ChangeNotifier`; `ListenableBuilder` en `main.dart` escucha	Al iniciar o cerrar sesión la app cambia de pantalla sola
Service / Repository	Estructural	`ApiService` concentra todas las llamadas REST; las pantallas no usan `http` directamente	Cambiar la URL o el formato de la API toca un solo archivo
Factory Method (DTO)	Creacional	`User.fromJson`, `Conversation.fromJson`, `ChatMessage.fromJson`	Conversión JSON → objeto tipado en un solo lugar
Chain of Responsibility (Middleware)	Comportamiento	`requireAuth` → `requireAdmin` → controlador en Express	Seguridad reutilizable; cada ruta declara qué permisos necesita
Strategy	Comportamiento	`generateReply` elige `replyWithOpenAI` o `replyWithClaude` según `AI_PROVIDER`	Cambiar de proveedor sin tocar rutas ni la app

Patrón arquitectónico en el cliente: MVVM simplificado. Las pantallas (`*_screen.dart`) son la Vista; `ApiService` actúa como ViewModel compartido que expone el estado de sesión; `models.dart` es el Modelo.

Mejora propuesta para versiones futuras: separar `ApiService` en `AuthRepository`, `ChatRepository` y `UserRepository` e inyectarlos con `provider` o `riverpod`, lo que facilita crear dobles de prueba para cada pantalla.

6. Justificación de los frameworks

Flutter se eligió para la app porque compila a código nativo desde una sola base de código en Dart, y Node.js con Express para el servidor por su rapidez de desarrollo y por tener SDK oficial de los proveedores de IA.

6.1 Framework móvil: comparación

Criterio	Flutter (elegido)	React Native	Kotlin nativo
Lenguaje	Dart	JavaScript / TypeScript	Kotlin

Criterio	Flutter (elegido)	React Native	Kotlin nativo
Plataformas con un código	Android, iOS, web, escritorio	Android, iOS	Solo Android
Interfaz	Motor propio, Material 3 incluido	Componentes nativos vía puente	Jetpack Compose
Recarga en caliente	Sí (hot reload)	Sí (Fast Refresh)	Parcial
Rendimiento	Compilado AOT a código máquina	Bueno, depende del puente JS	El más alto
Pruebas integradas	`flutter_test` (unitarias y de widgets)	Jest + librerías externas	JUnit + Espresso

Flutter gana por tener un solo código para Android e iOS, Material Design 3 con modo oscuro sin librerías extra, y herramientas de análisis y pruebas incluidas en el SDK.

6.2 Frameworks y librerías del proyecto

Componente	Framework / librería	Versión	Justificación
App móvil	Flutter / Dart	3.47.3 / 3.13.3	Multiplataforma, compilación nativa, hot reload
Cliente HTTP	`http`	^1.6.0	Paquete oficial de Dart, suficiente para REST JSON
Sesión local	`shared_preferences`	^2.5.5	Guarda el token JWT entre aperturas de la app
Servidor	Node.js + Express	22.11 / ^5.2.1	Minimalista, gran ecosistema, manejo nativo de `async`
Base de datos	SQLite (`node:sqlite`)	Integrado en Node 22.11	Sin instalar un servidor de BD; archivo único; claves foráneas. Reemplazó a `better-sqlite3`, cuyo binario nativo fallaba en Windows
Autenticación	`jsonwebtoken`	^9.0.3	Tokens sin estado, válidos 7 días
Hash de contraseñas	`bcryptjs`	^3.0.3	Hash con sal (costo 10), sin compilación nativa
IA (principal)	SDK `openai`, modelo `gpt-5.4-mini`	^7.23.0	ChatGPT; modelo rápido y económico para chat
IA (alternativa)	SDK `@anthropic-ai/sdk`, modelo `claude-opus-5`	^0.128.0	Se activa con `AI_PROVIDER=anthropic`
Configuración	`dotenv`	^18.0.3	Secretos en `.env`, fuera del código
CORS	`cors`	^2.8.6	Permite probar la API desde Flutter web

7. Esquema de pruebas

Las pruebas se organizan en cinco niveles, del análisis estático a la aceptación en el dispositivo; cada sprint termina solo si todos los niveles aplicables pasan.

7.1 Niveles de prueba

Nivel	Qué se prueba	Herramienta	Cuándo
Análisis estático	Errores de tipos, lints de estilo	`flutter analyze` + `flutter_lints`	Antes de cada commit
Unitarias / widgets	Pantalla de login sin sesión, modelos `fromJson`	`flutter test`	Antes de cada commit
Integración de API	Endpoints REST con base de datos de prueba aislada (`DB_PATH=./data/test.db`)	`curl` / Postman	Al cerrar cada sprint
Rendimiento	Tiempo de inicio de sesión	`curl -w %{time_total}`	Sprint 4
Aceptación (E2E)	Flujos completos en celular Android por USB	Dispositivo físico + `adb reverse`	Revisión de sprint

7.2 Casos de prueba

ID	Caso	Entrada	Resultado esperado	Estado
CP-01	Login correcto del administrador	`admin` / `Admin123!`	Token JWT y rol `admin`	Aprobado
CP-02	Login con contraseña incorrecta	`admin` / `x`	401 "Credenciales incorrectas"	Aprobado
CP-03	Registro de usuario nuevo	Nombre, usuario, clave de 8 caracteres	201 y sesión iniciada	Aprobado
CP-04	Crear usuario desde admin	`ana` / `secreto1`	201 con el usuario creado	Aprobado
CP-05	Editar usuario y cambiar rol	Nombre nuevo, rol `admin`	200 con datos actualizados	Aprobado
CP-06	Admin intenta borrarse a sí mismo	DELETE `/users/1`	400 "No puedes eliminar tu propia cuenta"	Aprobado
CP-07	Usuario normal accede al CRUD	GET `/users` con token de usuario	403 "Solo administradores"	Aprobado
CP-08	Borrar una conversación	DELETE `/conversations/1`	204 y mensajes borrados en cascada	Aprobado
CP-09	Borrar un usuario	DELETE `/users/2`	204 y sus conversaciones borradas	Aprobado
CP-10	Enviar mensaje sin créditos en OpenAI	"Hola"	502 "La cuenta de IA no tiene créditos"; el mensaje no queda guardado	Aprobado
CP-11	Enviar mensaje con créditos	"Hola, me llamo Kike" y luego "¿Cómo me llamo?"	La IA responde recordando el nombre	Pendiente (recargar créditos)
CP-12	Tiempo de inicio de sesión	Login por `localhost`	Menor a 0.5 s (medido 0.10 s; antes de la corrección IPv6: 2.03 s)	Aprobado

ID	Caso	Entrada	Resultado esperado	Estado
CP-13	Pantalla inicial sin sesión	Abrir la app	Se muestra "Chatbot IA" y el botón "Entrar"	Aprobado (`flutter test`)
CP-14	Sesión persistente	Cerrar y abrir la app	Entra directo al chat	Pendiente en dispositivo

8. Estrategia de versionamiento

El código se versiona con Git usando GitFlow simplificado, versiones semánticas (SemVer) y commits convencionales; cada sprint cierra con una etiqueta de versión.

8.1 Ramas

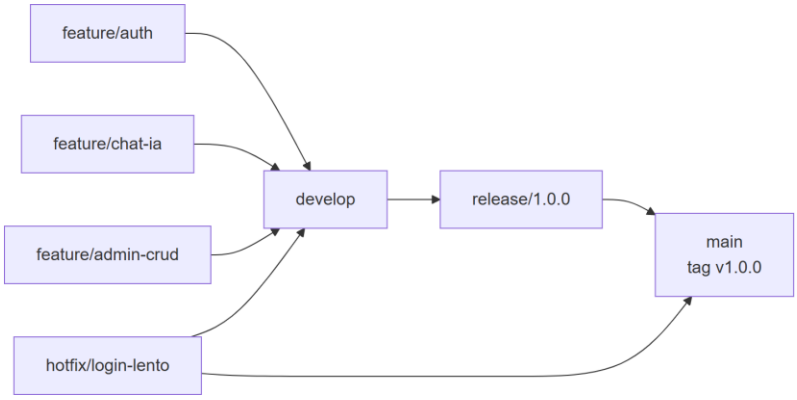


Figura 4. Flujo de ramas GitFlow simplificado

Rama	Propósito	Se une a
`main`	Versión estable entregable; cada commit lleva etiqueta	—
`develop`	Integración de funcionalidades terminadas	`release/*`
`feature/<nombre>`	Una historia de usuario por rama (ej. `feature/admin-crud`)	`develop`
`release/<versión>`	Pruebas finales y ajustes antes de publicar	`main` y `develop`
`hotfix/<nombre>`	Corrección urgente sobre `main` (ej. el login lento por IPv6)	`main` y `develop`

8.2 Versiones semánticas: MAYOR.MENOR.PARCHE+BUILD

- **MAYOR:** cambios incompatibles en la API (ej. cambiar el formato del login).
- **MENOR:** funcionalidad nueva compatible (ej. panel de administrador).
- **PARCHE:** correcciones (ej. mensaje de error de créditos).
- **BUILD:** número que aumenta en cada APK generado; Android lo usa como `versionCode`.

La app define su versión en `pubspec.yaml` (`version: 1.0.0+1`) y el servidor en `server/package.json` (`"version": "1.0.0"`). Ambos avanzan juntos cuando cambia la API.

8.3 Commits convencionales

Prefijo	Uso	Ejemplo
`feat:`	Nueva funcionalidad	`feat: CRUD de usuarios para administrador`
`fix:`	Corrección de error	`fix: escuchar en IPv6 para evitar login lento`
`docs:`	Documentación	`docs: instrucciones de adb reverse`
`test:`	Pruebas	`test: prueba de widget de login`
`refactor:`	Cambio interno sin nueva función	`refactor: proveedor de IA configurable`
`chore:`	Configuración y dependencias	`chore: puerto 8000 y tareas de VS Code`

8.4 Qué no se versiona

`server/.env` (claves y secretos), `server/data/` (base de datos), `server/node_modules/`, `build/` y `.dart_tool/`. En su lugar se versiona `server/.env.example` como plantilla sin claves.

9. Configuración del entorno

El entorno corre en Windows 11 con Flutter 3.47.3 y Node.js 22.11; el celular Android se conecta por USB y usa `adb reverse` para llegar al servidor en `localhost:8000`.

9.1 Herramientas utilizadas

Herramienta	Versión	Uso
Windows 11 Home	10.0.26200	Sistema operativo de desarrollo
Visual Studio Code	1.139.0	Editor, depuración y tareas (`.vscode/launch.json`, `tasks.json`)
Flutter SDK	3.47.3 (stable)	Framework de la app móvil
Dart SDK	3.13.3	Lenguaje de la app
Flutter DevTools	2.60.0	Inspección de widgets y rendimiento
Android SDK Platform-Tools (adb)	35.0.2	Instalar la app y redirigir puertos por USB
OpenJDK	17.0.15	Compilación Gradle de Android
Node.js / npm	22.11.0 / 10.9.0	Servidor backend y gestor de paquetes
SQLite	Integrado en Node (`node:sqlite`)	Base de datos
Git	2.46.0	Control de versiones
curl / Postman	—	Pruebas de la API
OpenAI Platform	—	Clave de API y facturación del modelo de IA

9.2 Parámetros del servidor (server/.env)

Variable	Valor usado	Descripción
PORT	8000	Puerto HTTP de la API
JWT_SECRET	Cadena aleatoria de 64 caracteres hex	Firma de los tokens de sesión
AI_PROVIDER	openai	Proveedor de IA (openai o anthropic)
OPENAI_API_KEY	Clave sk-proj-… (no se publica)	Acceso a ChatGPT
OPENAI_MODEL	gpt-5.4-mini	Modelo de lenguaje
ANTHROPIC_API_KEY	Vacía (opcional)	Acceso a Claude si se cambia de proveedor
ADMIN_USERNAME	admin	Usuario administrador creado al iniciar
ADMIN_PASSWORD	Admin123!	Contraseña inicial del administrador
DB_PATH	./data/chatbot.db	Archivo de la base de datos

9.3 Parámetros de la app

Parámetro	Valor	Dónde
API_BASE_URL	http://localhost:8000/api	--dart-define al ejecutar; valor por defecto en lib/config.dart
Permiso de red	android.permission.INTERNET	android/app/src/main/AndroidManifest.xml
Tráfico HTTP sin cifrar	usesCleartextTraffic="true"	AndroidManifest.xml (solo desarrollo)
Nombre visible	Chatbot IA	android:label en AndroidManifest.xml
Identificador	com.example.chatbotmovil	android/app/build.gradle.kts
Versión	1.0.0+1	pubspec.yaml
Tiempos límite	5 s sesión, 20 s peticiones, 3 min respuesta de IA	lib/api_service.dart

9.4 Pasos para levantar el entorno

1. Instalar dependencias del servidor: cd server y npm install.
2. Copiar server/.env.example a server/.env y colocar la clave de OpenAI.
3. Iniciar el servidor: npm start (muestra Servidor en http://localhost:8000).
4. Conectar el celular por USB con depuración USB activada y verificar con adb devices.
5. Ejecutar .\run_android.ps1, que corre adb reverse tcp:8000 tcp:8000 y flutter run --dart-define=API_BASE_URL=http://localhost:8000/api.
6. Iniciar sesión con admin / Admin123! y cambiar la contraseña desde Administrar usuarios.